PHY 3902 Experiment 4 Lab Report

Author(s): Michelle Johnstone, Patrick Geraghty
Affiliation: University of Ottawa
Date: November 2, 2025

1 Abstract

This study evaluates the performance of a proportional-integral-derivative (PID) control system for precise temperature regulation in an open system using a cost-effective experimental setup. The system comprises a thermistor for temperature sensing, a heating element, and an Arduino microcontroller to implement the PID algorithm. The PID approach dynamically adjusts the power supplied to the heating element by accounting for proportional (k_p), integral (k_i), and derivative (k_d) terms, enabling rapid and stable responses to temperature fluctuations caused by environmental changes. Experimental results demonstrated that the PID-controlled system maintained a stable temperature with a standard deviation of $\sigma = x$ and a percent error of XX%, significantly outperforming traditional on-off control methods. Furthermore, the data were used to calculate the specific heat capacities of aluminum ($\frac{J}{g^\circ C}$) and brass ($\frac{J}{g^\circ C}$), showcasing the system's utility in thermodynamic measurements. These findings underscore the potential of PID temperature control for applications requiring high precision, adaptability, and energy efficiency in dynamic environments.

2 Introduction

Temperature control systems are fundamental in many applications, yet the choice of control strategy significantly impacts performance and system longevity. Simple on-off control requires substantial hysteresis to prevent excessive cycling, which degrades circuit lifespan as the temperature range increases. Moreover, on-off controllers struggle to maintain set temperatures precisely and perform optimally only in thermally isolated systems requiring minimal power. In contrast, PID (proportional-integral-derivative) control excels in open systems exposed to environmental fluctuations, enabling rapid response to external temperature changes.

2.1 Proportional Control

The foundation of PID control is the proportional term, which supplies heater power proportional to the temperature error. The error is defined as the normalized difference between the desired and measured temperatures:

$$E = \frac{T_{\text{set}} - T_{\text{measured}}}{\text{band}} \tag{1}$$

This error modulates the heater power as a fraction of maximum power:

$$\text{Power} = (T_{\text{offset}} + k_p E)\, P_{\max} \tag{2}$$

where k_p is a proportional gain that controls the system's responsiveness. By reducing power as the system approaches the set temperature, proportional control minimizes overshoot. However, the resulting temperature-time response exhibits decaying oscillations, motivating the addition of integral and derivative terms.

2.2 Integral and Derivative Control

Integrating the error over time corrects steady-state deviations:

$$I = \int E \, dt \tag{3}$$

The integral term, scaled by the constant k_i, is added to eliminate persistent errors without inducing excessive oscillations. Similarly, derivative control responds to the rate of change of error:

$$D = -\frac{dE}{dt} = -\frac{\Delta E}{\Delta t} \tag{4}$$

where Δt corresponds to the control loop's execution time. The derivative gain k_d allows rapid reaction to external disturbances. The complete PID control law is:

$$\text{Power} = (T_{\text{offset}} + k_p E + k_i I + k_d D) \, P_{\max} \tag{5}$$

In systems with noisy temperature sensors, derivative control can amplify measurement noise, making low or zero values of k_d preferable.

2.3 Specific Heat Capacity Determination

The specific heat capacity of test materials was determined by analyzing heating curves under constant power. A timed heating and cooling cycle was implemented: the system supplied full power for x minutes, followed by x minutes of passive cooling. During the heating phase, the time required to achieve a 1°C temperature rise was recorded. The specific heat capacity is calculated as:

$$c = \frac{(P_{\text{heater}} - P_{\text{loss}}) \cdot \Delta t}{m \cdot \Delta T} \tag{6}$$

where P_{heater} is the supplied heater power, P_{loss} accounts for heat dissipation, m is the material mass in grams, Δt is the time interval, and $\Delta T = 1°C$ is the temperature increment.

Power supplied to the heater is controlled via a MOSFET, which converts a 5V PWM signal into a variable 12V power supply. The power delivered depends on the duty cycle L (0–255):

$$P = \frac{V_{\text{supply}}^2}{R_{\text{heater}}} \cdot \frac{L}{255} \tag{7}$$

Experimental accuracy is assessed using the standard percent error formula:

$$\text{Percent Error} = \frac{|\text{Experimental} - \text{Theoretical}|}{\text{Theoretical}} \times 100\% \tag{8}$$

3 Methods

3.1 Experimental Setup

The experimental apparatus consisted of a heating element controlled via a MOSFET, a thermistor for temperature measurement, and a microcontroller running custom firmware to implement temperature control algorithms. A wiring diagram of the complete setup is provided in Figure 1.

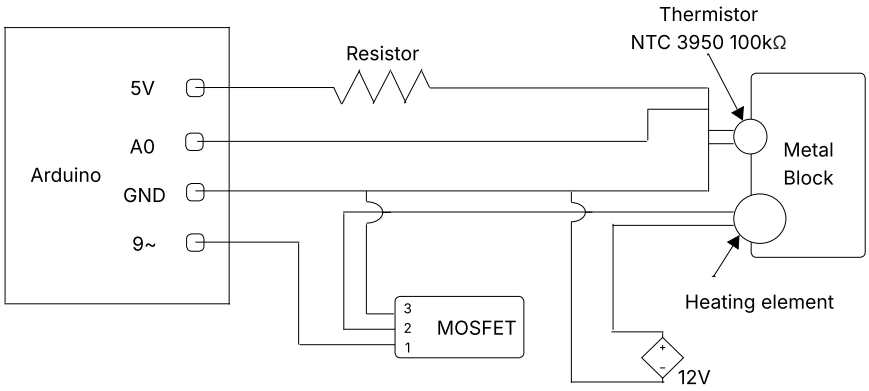


Figure 1: Wiring diagram of the experimental setup used for PID temperature control.

3.2 On-Off Control Baseline

To establish a baseline response characteristic of the system, an on-off control algorithm was implemented. The heating element operated at maximum power until the thermistor registered the set temperature of 65°C, at which point the heater was disabled until the temperature dropped to the low temperature limit. Temperature readings from the thermistor were recorded at approximately one-second intervals and plotted as a function of time. The expected behavior was characterized by a positive linear increase to a peak above the set temperature, followed by a steeper decline back to the lower temperature threshold, confirming proper circuit functionality.

3.3 Proportional Control Implementation

Following the baseline characterization, proportional control was implemented using equation (2) to modulate heating power based on the error between actual and set temperatures. This approach reduced temperature oscillations by preventing full on-off cycling and better maintained the temperature within the desired hysteresis band. The proportionality constant k_p was initialized to 0.1 and incrementally increased until measurable performance improvements were observed. Despite these improvements, the system still exhibited a significant error relative to the set temperature and continued to overshoot during each oscillation cycle.

3.4 System Response Time Characterization

To optimize the integral control parameters, the system's reaction time was determined using a step response method. The reaction time was defined as the time required for the temperature to reach 63% of the total step change from baseline to target temperature [1]. A step input was applied by enabling the heating element to full power via the MOSFET while continuously recording temperature measurements. The initial temperature was measured using the thermistor, and the target temperature was set to 5°C above this baseline value. The time to reach the 63% threshold was recorded as the system's response time. In accordance with laboratory guidelines [3], the integral action time constant was set to twice the measured response time to ensure adequate system stabilization.

3.5 Integral Control Refinement

Integral control was implemented to minimize the error and reduce the oscillatory behavior observed with proportional control alone. The integral constant k_i was initially set to approximately 0.0001 to minimize the speed and magnitude of integral corrections, thereby reducing the risk of integral windup [4] - a phenomenon in which excessive error accumulation during actuator saturation leads to overshooting and instability. The integral constant was then gradually increased until the desired system performance was achieved without overcorrection or instability.

3.6 Derivative Control Optimization

Derivative control was implemented to improve transient response and combat sudden environmental temperature changes. However, derivative action amplifies high-frequency noise in thermistor measurements and measurement errors, potentially inducing unwanted oscillations. Therefore, the derivative constant k_d was initialized to 0.01 and systematically decreased until the system ceased overreacting to measurement noise while maintaining its ability to handle step changes in environmental conditions. Although derivative control is particularly beneficial for systems with rapidly changing dynamics, its implementation in this relatively stable thermal system required careful tuning to avoid amplifying measurement artifacts.

3.7 Thermal Characterization

Temperature-time response graphs were acquired at two different set temperatures. The voltage required to maintain each temperature was determined from the duty cycle of the MOSFET at the stabilized portion of each response curve using equation 7 to calculate the heat loss from the block. Subsequently, the heating element was operated at full power using Listing 11 until a predetermined maximum temperature was reached. The time and power required to raise the temperature by 1°C were extracted from these measurements. These values were then substituted into equation 6 to determine the specific heat capacity of the test block. Measurements at the two different temperatures allowed assessment of temperature-dependent variations in specific heat capacity and estimation of measurement uncertainty.

4 Results and Discussion

4.1 On-Off Control Performance

The circuit was initially tested using a basic on-off control strategy to maintain a target temperature of 55°C. As shown in Figure 2, the system's temperature increased steadily over time and eventually leveled off near the target temperature.

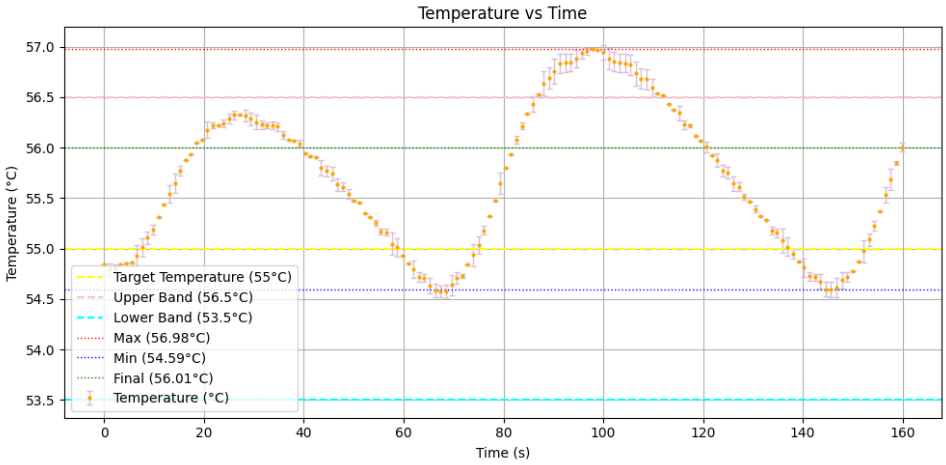


Figure 2: Temperature vs. time for the on-off control circuit regulating an aluminum block to a target of 55°C. The system shows an oscillating temperature curve, with local minima just below the set temperature, demonstrating the basic performance of simple on-off control.

When hysteresis control was introduced, with a 3°C range around the target temperature, the system exhibited a pattern of temperature fluctuations, as shown in Figure 3.

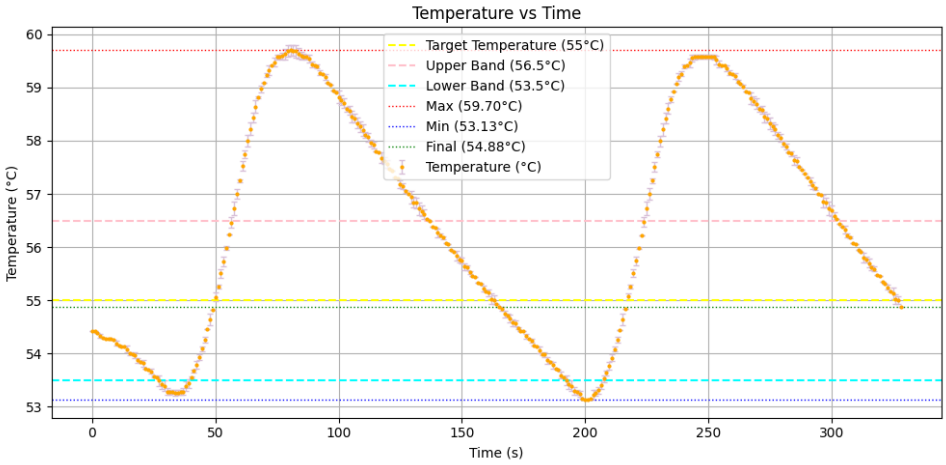


Figure 3: Temperature vs. time for the on-off circuit with a 3°C hysteresis band and a 55°C target. The plot shows characteristic oscillations as the heater cycles on and off, with sharp increases during heating phases and less pronounced drops during cooling.

The temperature increased steadily during heating but exceeded the upper limit of the range before cooling down rapidly. The temperature then dropped below the lower limit of the range before the heater turned back on. This uneven heating and cooling

pattern suggests that the aluminum block loses heat slower than it absorbs heat from the heater. These results highlight the limited accuracy of simple on-off control methods for maintaining a stable temperature in systems like this. These measurements provided a baseline for comparing the performance of more advanced control methods.

4.2 Proportional Control Implementation

Proportional control was implemented to improve temperature regulation. This method adjusts the heater's power based on how far the current temperature is from the target. Figure 4 shows the system's behavior with a proportionality factor (K_p) of 0.3, an offset of 0.1, and a temperature range of 3°C. This approach reduced the size of temperature overshoots and made the system more stable, although the offset value was not ideal for keeping the temperature close to the target.

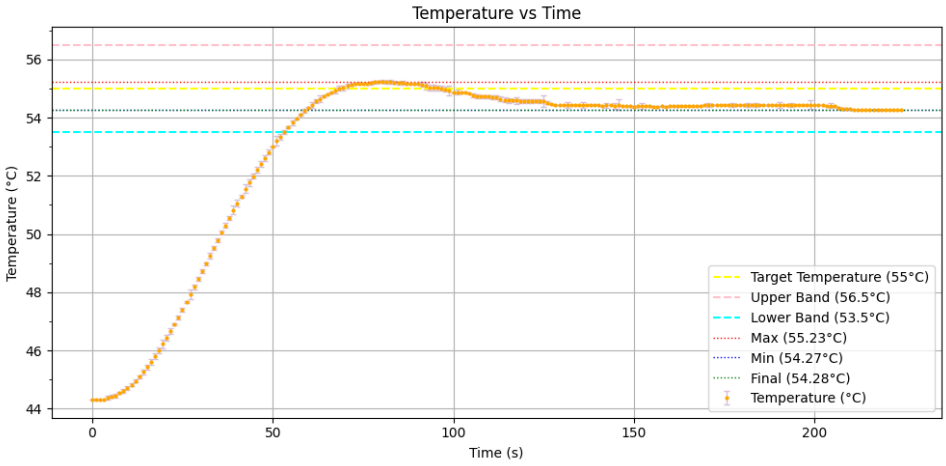


Figure 4: Temperature vs. time for proportional control with a 3°C band, offset of 0.1, and $K_p = 0.3$ for an aluminum block at 55°C. The system exhibits reduced overshoot and improved stability compared to on-off control, but the offset is not fully optimized.

A further adjustment to the offset, increasing it to 0.165, improved the performance, as shown in Figure 5.

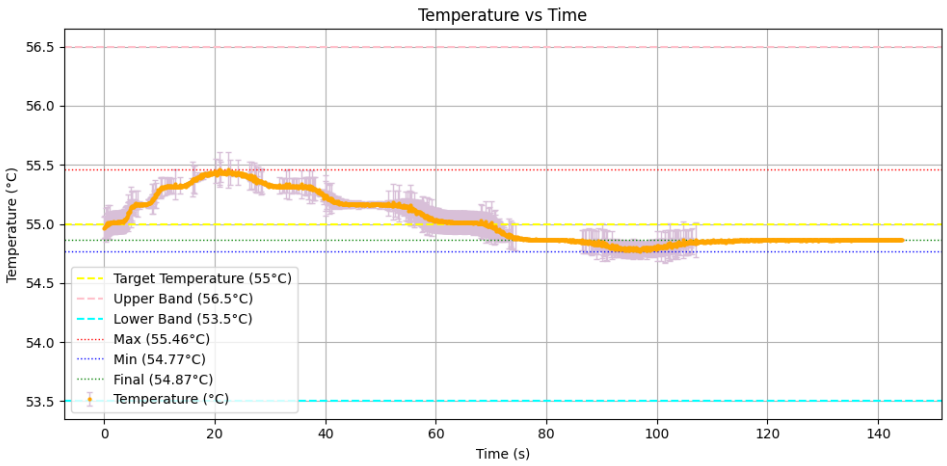


Figure 5: Temperature vs. time for proportional control with a 3°C band, offset of 0.165, and $K_p = 0.3$ for an aluminum block at 55°C. Improved offset tuning results in better temperature regulation and smaller fluctuations.

The system was able to stay within the target range for longer periods, and the temperature fluctuations became smaller over time. This behavior is typical of systems using proportional control, where the heater's power is adjusted based on the difference between the current temperature and the target, allowing for smoother changes compared to simple on-off control.

The temperature range was then reduced to 1°C while keeping the same optimized settings ($K_p = 0.3$, offset $= 0.165$). This adjustment allowed the system to regulate the temperature more precisely, as shown in Figure 6.

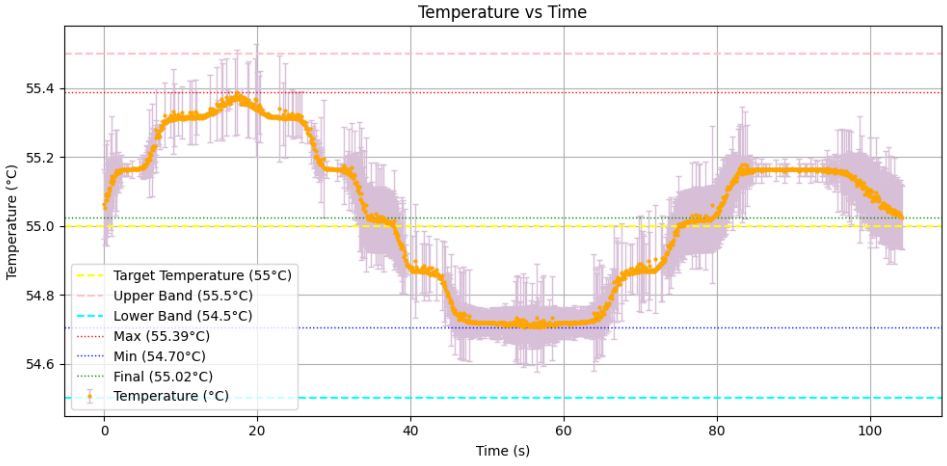


Figure 6: Temperature vs. time for proportional control with a 1°C band, offset of 0.165, and $K_p = 0.3$ for an aluminum block at 55°C. Narrowing the control band allows for more precise temperature regulation, though a small error remains.

However, the system still exhibited a small but consistent error, where the temperature was slightly higher or lower than the target. This occurred because proportional control requires a small difference between the actual and target temperatures to maintain the heater's power and balance heat loss to the environment.

4.3 Proportional-Integral Control

To address the consistent temperature error observed with proportional control, an integral control component was added. Figures 7, and 8, show the performance of the proportional-integral (PI) controller with $K_p = 0.3$, $K_i = 0.00001$, an offset of 0.165, and a 1°C range, applied to the aluminum block at a target temperature of 55°C. The integral control component works by gradually correcting any long-term differences between the current temperature and the target.

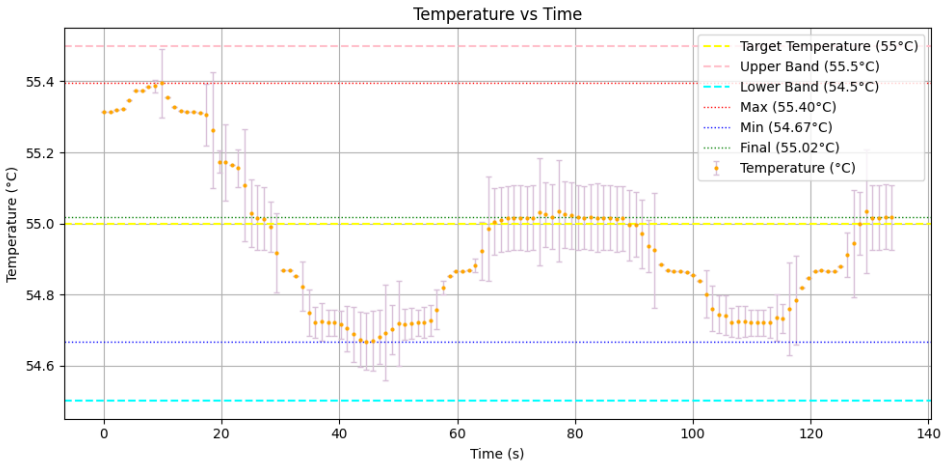


Figure 7: Temperature vs. time for PI control with a 1°C band, offset of 0.165, $K_p = 0.3$, and $K_i = 0.00001$ for an aluminum block at 55°C. The addition of integral action reduces steady-state error and improves long-term stability.

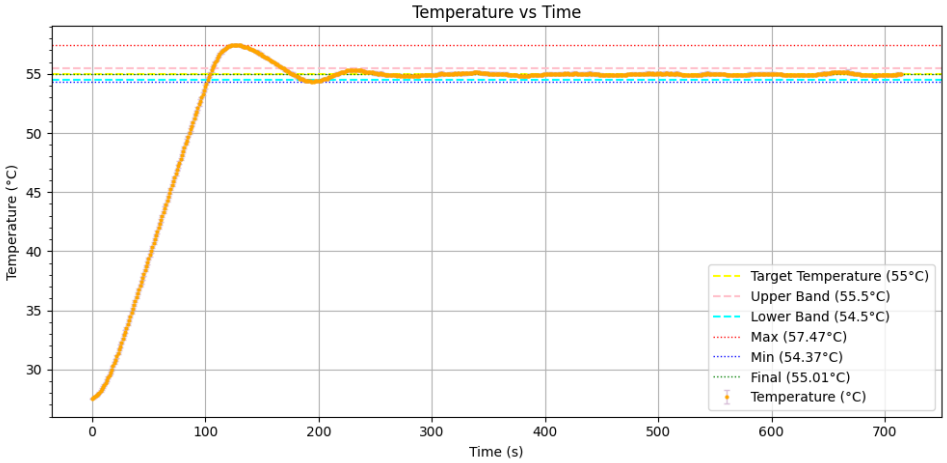


Figure 8: Temperature vs. time for PI control (Aluminum): $T_{\text{set}} = 55$°C, band=1, $K_p = 0.3$, $K_i = 0.00001$, offset=0.165. The plot demonstrates the system's ability to maintain temperature near the setpoint over time.

This adjustment allowed the system to reduce the consistent error and stabilize the temperature closer to the desired value. Over time, the size of the temperature fluctuations decreased until the system maintained a more stable performance near the target.

At a higher target temperature of 65°C, the PI controller with an increased integral gain ($K_i = 0.001$) and an offset of 0.25 showed effective performance, as illustrated in

Figure 9.

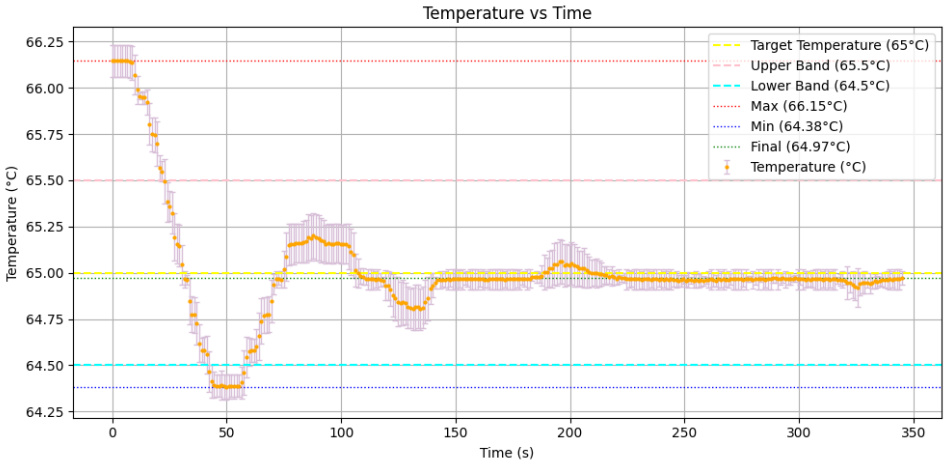


Figure 9: Temperature vs. time for PI control at a higher setpoint (65°C) with $K_p = 0.3$, $K_i = 0.001$, $K_d = 0.0$, and offset=0.25. The increased integral gain enables faster correction of errors, demonstrating adaptability to different target temperatures.

The higher integral gain allowed faster correction of temperature errors, demonstrating that the system could adapt effectively to different target temperatures.

4.4 Full PID Control and Derivative Action

The final step in the control strategy involved adding a derivative control component, completing the proportional-integral-derivative (PID) system. Derivative control helps the system respond more quickly to sudden changes in temperature by predicting how the temperature is likely to change. Figure 10 shows the system's behavior with the derivative gain set to zero ($K_d = 0.0$), effectively functioning as a PI controller for comparison.

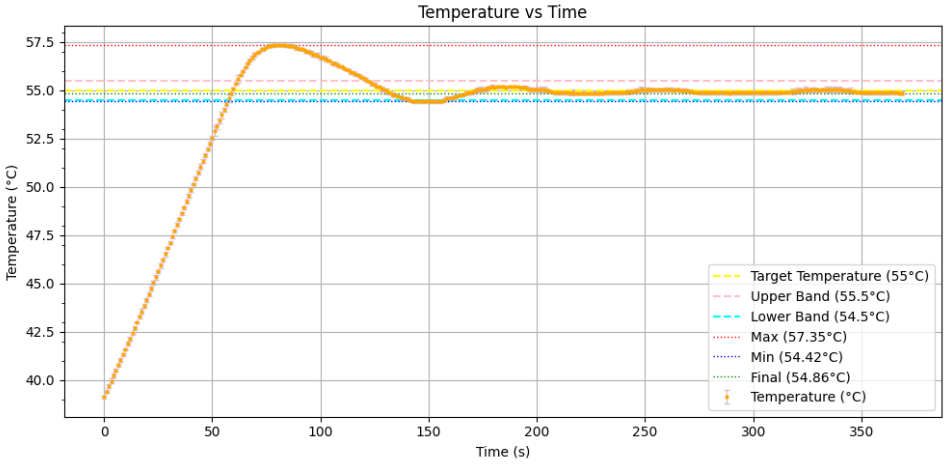


Figure 10: Temperature vs. time for PID control (Aluminum): $T_{\text{set}} = 55$°C, band=1, $K_p = 0.3$, $K_i = 0.00001$, $K_d = 0.0$, offset=0.165. With $K_d = 0$, the system acts as a PI controller, providing a baseline for comparison with full PID control.

When derivative control was activated ($K_d = 0.00001$), the system demonstrated a much faster response to sudden temperature changes, as shown in Figure 11.

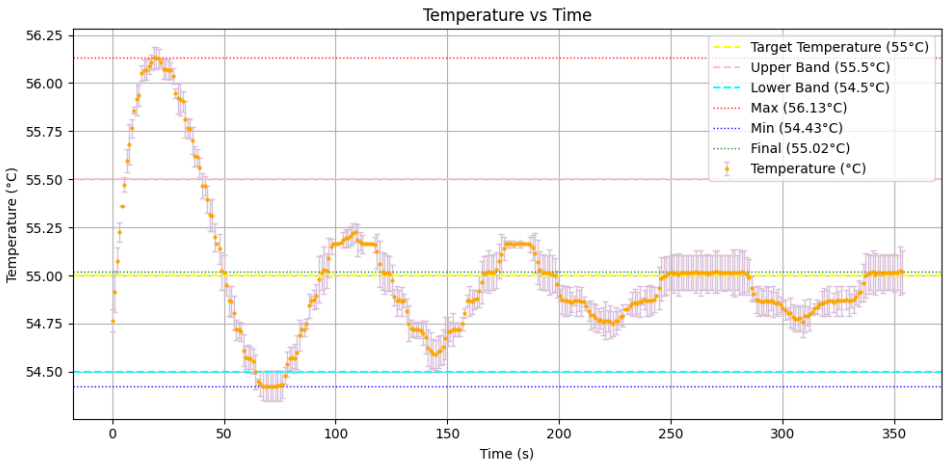


Figure 11: Temperature vs. time for PID control (Aluminum): $T_{\text{set}} = 55°\text{C}$, band=1, $K_p = 0.3$, $K_i = 0.00001$, $K_d = 0.00001$, offset=0.165. Activating derivative control results in a faster response to sudden temperature changes, maintaining a much tighter oscilation around the setpoint.

For instance, during testing, the temperature sensor was removed from the aluminum, causing it to detect a rapid drop in temperature. The derivative control responded by immediately increasing the heater power to 100%, highlighting its ability to handle unexpected changes. This response occurred with $K_d = 0.001$, showing that even small derivative gains have a significant effect on how the system reacts to sudden events.

However, the derivative control also introduced a drawback: it amplified small fluctuations in the temperature readings caused by sensor noise. This resulted in slightly more variability in the heater's power during normal operation, which partially reduced the benefits of the faster response to temperature changes.

The PID controller was also tested on a brass block at 65°C, using $K_p = 0.3$, $K_i = 0.001$, $K_d = 0.001$, and an offset of 0.25, as shown in Figure 12.

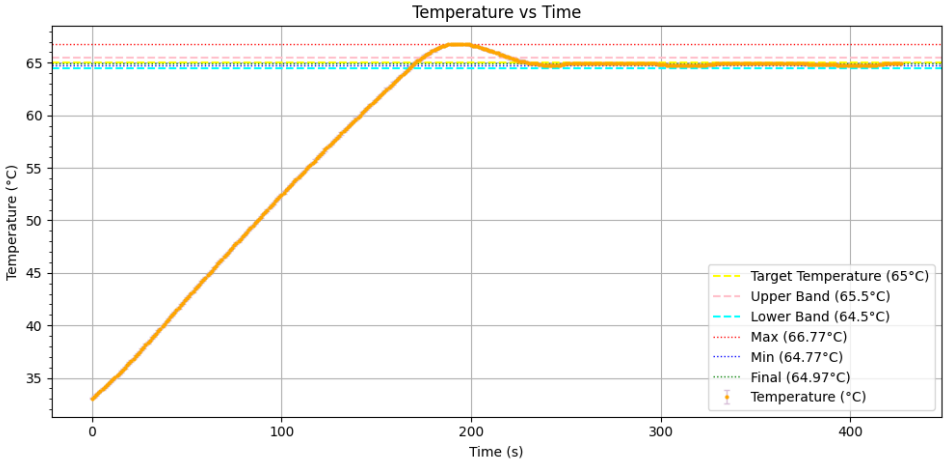


Figure 12: Temperature vs. time for PID control (Brass): $T_{\text{set}} = 65°\text{C}$, band=1, $K_p = 0.3$, $K_i = 0.001$, $K_d = 0.001$, offset=0.25. The PID controller successfully regulates the temperature of a brass block, demonstrating adaptability to different materials.

The PID controller successfully regulated the temperature of the brass block, demonstrating its ability to adapt to materials with different thermal properties.

4.5 Specific Heat Capacity Measurements

The specific heat capacity determinations yielded results with substantial deviations from accepted literature values. For the aluminum block (24.64 ± 0.005 g), the calculated specific heat capacity was $1.8\,\frac{\text{J}}{\text{g}\cdot^\circ\text{C}}$, representing a 100% error relative to the accepted value of $0.9\,\frac{\text{J}}{\text{g}\cdot^\circ\text{C}}$ [2]. Similarly, the brass block (72.35 ± 0.005 g) yielded $0.8\,\frac{\text{J}}{\text{g}\cdot^\circ\text{C}}$ compared to the literature value of $0.4\,\frac{\text{J}}{\text{g}\cdot^\circ\text{C}}$ [2], again corresponding to a 100% error.

While it was noted that aluminum measurements typically exceed theoretical values in this experimental setup, the magnitude of the discrepancies suggests systematic error rather than random uncertainty. Potential error sources include: (1) inaccuracies in PWM-to-power conversion calculations, (2) errors in MOSFET switching efficiency modeling, (3) miscalibration of the energy delivery measurement, (4) unaccounted heat losses to the environment during the heating interval, or (5) noise amplification in the data acquisition chain affecting temperature change rate calculations.

Experimental limitations prevented comprehensive error analysis through extended data collection. Hardware failures during the experiment, including damaged components requiring replacement, significantly compressed the available testing time. A more extensive measurement campaign spanning a wider temperature range would have facilitated identification of whether errors exhibited systematic trends or random scatter, thereby isolating the dominant uncertainty source. The consistent overestimation for both materials suggests a systematic calibration error in the power delivery measurement or energy calculation algorithm rather than material-specific measurement artifacts.

5 Conclusion

In conclusion, the Arduino-based PID temperature control system effectively demonstrated progressive improvements in stability and response across the ON–OFF, P, PI, and PID configurations. The PI controller achieved the most stable performance, maintaining the target temperature with the smallest standard deviation and least noise reactivity, validating its theoretical advantages. However, the experimentally determined specific heat capacities for aluminum and brass were significantly higher than accepted values, primarily due to systematic errors. These could have included code-related issues such as the chosen constants or improper power scaling of the PWM signals, circuitry losses through the MOSFET and wiring, limited thermal insulation, and sensor inaccuracies from the thermistor. Had there been more time for the experiment it could have proven beneficial to incorporate direct voltage and current measurements for accurate power determination, improved sensor calibration, and refined control code to eliminate offset bias. Despite these discrepancies, the experiment successfully demonstrated the principles and effectiveness of PID-based temperature regulation.

References

[1] Wikipedia contributors. Step response. `https://en.wikipedia.org/wiki/Step_response`. Accessed: 2025-10-29.

[2] Engineers Edge. Specific heat capacity of metals table chart. `https://www.engineersedge.com/materials/specific_heat_capacity_of_metals_13259.htm`. Accessed: 2025-11-02.

[3] Michel Godin. Experiment 4: Servo control loops (feedback control systems) - pid temperature controller. Course handout, Brightspace learning platform. Accessed: 2025-10-29.

[4] David Peterson. Integral windup method in pid control. https://control.com/technical-articles/intergral-windup-method-in-pid-control/. Accessed: 2025-10-29.

A Supplementary Code

Python Scripts

```python
import json
import os
import sys
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
import pandas as pd
from dataclasses import dataclass
from typing import Optional
from utils import BaseConfig, BaseConstants


@dataclass
class TempConstants(BaseConstants):
    TSET: float = 55.0  # Default target temperature in Celsius
    BAND: float = 3.0  # Acceptable band around target temperature
    COLS: tuple = ("time(s)", "temp_mean(C)", "temp_stddev(C)", "
        output_power")


@dataclass
class TempConfig(BaseConfig):
    DATA_DIR: str = os.path.abspath(os.path.join("..", "data", "
        PID_data"))


CONFIG = TempConfig()
CONSTANTS = TempConstants()



def get_data_path(file_name: str, config: TempConfig) -> str:
    """Return the absolute path to the data file."""
    return os.path.abspath(os.path.join(os.path.dirname(__file__),
        config.DATA_DIR, file_name))


def check_data_file(file_path: str, required_cols) -> Optional[pd.
    DataFrame]:
    if not os.path.isfile(file_path):
        print(f"Data file {file_path} does not exist.")
        return None
    data = pd.read_csv(file_path)
```

```
    if missing := [col for col in required_cols if col not in data.
        columns]:
        print(f"Missing column(s) in data file: {', '.join(missing)}")
        return None
    return data


def plot_temperature_data(file_name: str, band: Optional[float] = None,
    tset: Optional[float] = None):
    # Set default band and tset if not provided
    band = band if band is not None else CONSTANTS.BAND
    tset = tset if tset is not None else CONSTANTS.TSET

    # Retrieve and validate data
    file_path = get_data_path(file_name, CONFIG)
    data = check_data_file(file_path, CONSTANTS.COLS)
    if data is None:
        return

    # Plotting
    plt.figure(figsize=(10, 5))
    plt.errorbar(
        data["time(s)"],
        data["temp_mean(C)"],
        yerr=data["temp_stddev(C)"],
        label="Temperature ( C )",
        color="orange",
        markersize=2,
        fmt="o",
        ecolor="thistle",
        elinewidth=1,
        capsize=2,
    )
    plt.title("Temperature vs Time")
    plt.xlabel("Time (s)")
    plt.ylabel("Temperature ( C )")
    # Add target and band lines
    plt.axhline(
        y=tset,
        color="yellow",
        linestyle="--",
        label=f"Target Temperature ({tset} C )",
    )
    if band:
        plt.axhline(
            y=tset + band / 2,
            color="pink",
            linestyle="--",
            label=f"Upper Band ({tset + band / 2} C )",
        )
        plt.axhline(
            y=tset - band / 2,
            color="cyan",
            linestyle="--",
            label=f"Lower Band ({tset - band / 2} C )",
        )
    # Add max, min, final value lines
    temp_vals = data["temp_mean(C)"]
```

```
    max_val = temp_vals.max()
    max_pos = temp_vals.to_numpy().argmax()
    # Determine min value after max position to avoid initial low temps from heating phase
    if max_pos < len(temp_vals) - 1:
        min_after_max = temp_vals.iloc[max_pos+1:]
        min_val = temp_vals.iloc[-1] if min_after_max.empty else min_after_max.min()
    else:
        min_val = temp_vals.iloc[-1]
    final_val = float(temp_vals.iloc[-1])
    # Draw horizontal lines for max, min, and final values
    plt.axhline(y=max_val, color="red", linestyle=":", linewidth=1, label=f"Max ({max_val:.2f} C )")
    plt.axhline(y=min_val, color="blue", linestyle=":", linewidth=1, label=f"Min ({min_val:.2f} C )")
    plt.axhline(y=final_val, color="green", linestyle=":", linewidth=1, label=f"Final ({final_val:.2f} C )")

    plt.legend()
    plt.grid(True)
    # Customize y-axis ticks to add minor ticks
    ax = plt.gca()
    ax.yaxis.set_minor_locator(ticker.MultipleLocator(1))
    ax.yaxis.set_major_locator(ticker.AutoLocator())
    plt.tight_layout()
    # Save plot to out directory
    out_dir = os.path.abspath(os.path.join(os.path.dirname(__file__), "../out"))
    os.makedirs(out_dir, exist_ok=True)
    out_path = os.path.join(out_dir, file_name.replace(".csv", ".png"))
    plt.savefig(out_path)
    print(f"Plot saved to {out_path}")
    plt.show()


if __name__ == "__main__":
    # Allow CLI usage: python temp_plot.py <csv_file> [band] [tset] [auto]
    auto = False
    fname = None
    band = None
    tset = None
    if len(sys.argv) > 1:
        # Check if 'auto' is present as an argument
        if any(arg.lower() == "auto" for arg in sys.argv[1:]):
            auto = True
        else:
            fname = sys.argv[1]
            band = float(sys.argv[2]) if len(sys.argv) > 2 else None
            tset = float(sys.argv[3]) if len(sys.argv) > 3 else None
    else:
        # Prompt for file name or auto
        mode = input("Enter 'auto' to plot all from manifest, or press Enter for manual: ").strip().lower()
        if mode == "auto":
            auto = True
        else:
```

```
            fname = mode if mode else input("File name: ")
            band_input = input("Band (optional, press Enter to skip): "
                )
            tset_input = input("Tset (optional, press Enter to skip): "
                )
            band = float(band_input) if band_input else None
            tset = float(tset_input) if tset_input else None

    if auto:
        # Load manifest.json and run plot_temperature_data for each
            entry
        manifest_path = os.path.join(CONFIG.DATA_DIR, "manifest.json")
        with open(manifest_path, "r", encoding="utf-8") as f:
            manifest = json.load(f)
        for entry in manifest:
            fname = entry.get("filename")
            tset = entry.get("Tset")
            band = entry.get("band")
            print(f"Plotting {fname} (Tset={tset}, band={band})...")
            plot_temperature_data(fname, band, tset)
    else:
        plot_temperature_data(fname, band, tset)
```

Listing 1: temp_plot.py: Python script to analyze and plot temperature data.

```
import sys
import os
import json
import numpy as np
import matplotlib.pyplot as plt
from utils.data_processor import load_experiment_data, load_pid_data,
    find_stable_power_window, mean_output_power
from utils.specific_heat_calc import output_power_to_physical_power,
    calc_specific_heat

# Constants
R_heater = 15.7  # ohms
V_SUPPLY = 12.0


def get_Tset():
    try:
        Tset = float(input("Enter Tset (deg C): "))
    except Exception:
        print("Invalid input. Using default Tset=30.0")
        Tset = 30.0
    return Tset

def analyze_file(data_path, tset, m_block, plot=True):
    """
    Analyze experiment data to find energy required to raise
        temperature by 1 degree.

    Args:
        data_path: Path to CSV data file.
        tset: Target temperature setpoint (deg C).
        m_block: Mass of the block (grams).
        plot: Whether to plot the temperature data.
```

```
    """
    data = load_experiment_data(data_path)
    power = data['power(W)'].to_numpy(dtype=float)
    time = data['time(s)'].to_numpy(dtype=float)
    temp = data['temp_mean(C)'].to_numpy(dtype=float)
    tset_plus = tset + 1.0
    idx_start = np.argmax(temp >= tset)
    idx_end = np.argmax(temp >= tset_plus)
    # Validate indices
    if temp[idx_start] < tset or temp[idx_end] < tset_plus or idx_start
        == idx_end:
        print(f"Could not find valid temperature interval for Tset={
            tset} and Tset+1.")
        return None
    t_start = time[idx_start]
    t_end = time[idx_end]
    dt = t_end - t_start
    mean_power = np.mean(power[idx_start:idx_end+1])
    energy = mean_power * dt  # Joules
    if plot:
        plt.figure(figsize=(10,5))
        plt.plot(time, temp, label='Temperature')
        plt.axhline(tset, color='green', linestyle='--', label=f'Tset =
            {tset:.2f} C')
        plt.axhline(tset_plus, color='red', linestyle='--', label=f'
            Tset+1 = {tset_plus:.2f} C')
        plt.axvspan(t_start, t_end, color='orange', alpha=0.2, label='
            Interval')
        plt.xlabel('Time (s)')
        plt.ylabel('Temperature (C)')
        plt.title('Temperature Rise by 1 Degree')
        plt.legend()
        plt.tight_layout()
        plt.show()
    return {'energy': energy, 'dt': dt, 'mean_power': mean_power, '
        t_start': t_start, 't_end': t_end}

def auto_mode():
    """
    Automatic mode: process multiple data files as specified in
        manifest.json.
    """
    manifest_path = os.path.join('experiment_4', 'data', 'c_data', '
        manifest.json')
    with open(manifest_path, 'r') as f:
        manifest = json.load(f)
    for entry in manifest:
        print(f"\n--- Analyzing {entry['filename']} at Tset={entry['
            Tset']} ---")
        data_path = os.path.abspath(
            os.path.join('experiment_4', 'data', 'c_data', entry['
                filename'])
        )
        pid_path = os.path.abspath(
            os.path.join('experiment_4', 'data', 'PID_data', entry['
                PID_data'])
        )
        tset = entry['Tset']
```

```
        m_block = entry['m_grams']
        # Main experiment analysis
        result = analyze_file(data_path, tset, m_block, plot=True)
        if result is None:
            continue
        # PID data analysis
        pid_df = load_pid_data(pid_path)
        start_idx, end_idx = find_stable_power_window(pid_df, tset,
            window=30, tol=0.2)
        if start_idx is None:
            print(f"No stable window found near Tset={tset} in PID data
                .")
            continue
        avg_output_power = mean_output_power(pid_df, start_idx, end_idx
            )
        avg_physical_power = output_power_to_physical_power(
            avg_output_power, v_supply=V_SUPPLY, r_heater=R_heater)
        print(f"Stable PID output_power: {avg_output_power:.2f} (0-255)
            , Physical Power: {avg_physical_power:.3f} W")
        # Specific heat calculation
        c = calc_specific_heat(result['energy'], m_block, delta_t=1.0)
        print(f"Specific heat capacity: {c:.3f} J/(g*C)")

def main():
    if len(sys.argv) > 1 and sys.argv[1] == 'auto':
        auto_mode()
    else:
        # Interactive/manual mode
        m_block = 24.63  # grams (default)
        Tset = get_Tset()
        analyze_file('experiment_4/data/c_data/
            specific_heat_capacity_data_aluminium.csv', Tset, m_block,
            plot=True)

if __name__ == "__main__":
    main()
```

Listing 2: specific_heat_capacity.py: Python script to calculate specific heat capacity from temperature data.

Util Classes

```
#!/usr/bin/env python3
"""
Constants Module

This module provides constants used across multiple physics experiments
    .
"""

from dataclasses import dataclass


@dataclass
class BaseConstants:
    """Base constants used across multiple experiments."""
    V_SUPPLY: float = 5.0            # Supply voltage (V)
```

```
    R_FIXED: float = 100000.0            # Fixed resistor value (ohms)
    ANALOG_RESOLUTION: int = 1023    # 10-bit ADC resolution


@dataclass
class BaseConfig:
    """Base configuration for file paths."""
    OUT_DIR: str = '../out/'
```

Listing 3: constants.py: Shared constants

```
import numpy as np
import pandas as pd


def load_experiment_data(filename):
    """Load main experiment CSV data."""
    return pd.read_csv(filename)


def load_pid_data(filename):
    """Load PID data CSV (expects columns: time(s), temp_mean(C),
       output_power, etc)."""
    return pd.read_csv(filename)


def find_stable_power_window(pid_df, tset, window=30, tol=0.2):
    """
    Find a window where temperature is stable near tset (within tol deg
        C for at least 'window' seconds).
    Returns (start_idx, end_idx) or (None, None) if not found.
    """
    temp = pid_df['temp_mean(C)'].to_numpy(dtype=float)
    time = pid_df['time(s)'].to_numpy(dtype=float)
    mask = np.abs(temp - tset) < tol
    if not np.any(mask):
        return None, None
    # Find longest contiguous region where mask is True and duration >=
        window
    best = (None, None, 0)
    i = 0
    while i < len(mask):
        if mask[i]:
            j = i
            while j + 1 < len(mask) and mask[j + 1]:
                j += 1
            if time[j] - time[i] >= window and (j - i) > (best[2]):
                best = (i, j, j - i)
            i = j + 1
        else:
            i += 1
    if best[0] is not None:
        return best[0], best[1]
    return None, None


def mean_output_power(pid_df, start_idx, end_idx):
    return pid_df['output_power'].iloc[start_idx:end_idx+1].mean()
```

Listing 4: data_processor.py: Util functions for interpretting temperature data.

```python
def output_power_to_physical_power(output_power, v_supply=12.0,
    r_heater=15.7):
    """
    Convert output_power (0-255) to actual power in Watts.
    """
    v = output_power * v_supply / 255.0
    i = v / r_heater
    p = v * i
    return p

def calc_specific_heat(energy_joules, mass_grams, delta_t=1.0):
    """
    Calculate specific heat capacity in J/(g*C).
    """
    return energy_joules / (mass_grams * delta_t)
```

Listing 5: specific_heat_calc.py: Util functions for calculting specific heat capacities from temperature data.

Data Manifests

```json
[
  {
    "filename": "specific_heat_capacity_data_aluminium.csv",
    "PID_data": "live_plot_20251029_163401.csv",
    "Tset": 65,
    "m_grams": 24.63
  },
  {
    "filename": "specific_heat_capacity_data_aluminium.csv",
    "PID_data": "live_plot_20251028_162354.csv",
    "Tset": 55,
    "m_grams": 24.63
  },
  {
    "filename": "specific_heat_capacity_data_brass.csv",
    "PID_data": "live_plot_20251029_170448.csv",
    "Tset": 65,
    "m_grams": 79.14
  }
]
```

Listing 6: c_data/manifest.json: Json array for specific heat capacity data files and the respective constants required to determine the specific heat capacity.

```json
[
  {
    "filename": "baseline.csv",
    "Tset": 55,
    "band": 3,
    "control_type": "ON-OFF",
    "notes": "Initial baseline run"
  },
```

```
  {
    "filename": "simple_hysteresis.csv",
    "Tset": 55,
    "band": 3,
    "control_type": "ON-OFF",
    "notes": "Hysteresis only"
  },
  {
    "filename": "proportional_0.1_offset.csv",
    "Tset": 55,
    "band": 3,
    "control_type": "proportional",
    "Offset": 0.1,
    "Kp": 0.3,
    "notes": "Aluminium"
  },
  {
    "filename": "proportional_0.165_offset.csv",
    "Tset": 55,
    "band": 3,
    "control_type": "proportional",
    "Offset": 0.165,
    "Kp": 0.3,
    "notes": "Aluminium"
  },
  {
    "filename": "proportional_0.165_offset_1_band.csv",
    "Tset": 55,
    "band": 1,
    "control_type": "proportional",
    "Offset": 0.165,
    "Kp": 0.3,
    "notes": "Aluminium"
  },
  {
    "filename": "integral_0.165_offset_1_band.csv",
    "Tset": 55,
    "band": 1,
    "control_type": "PI",
    "Offset": 0.165,
    "Kp": 0.3,
    "Ki": 0.00001,
    "notes": "Aluminium"
  },
  {
    "filename": "live_plot_20251027_153510_integral.csv",
    "Tset": 55,
    "band": 1,
    "control_type": "PI",
    "Offset": 0.165,
    "Kp": 0.3,
    "Ki": 0.00001,
    "notes": "Aluminium"
  },
  {
    "filename": "live_plot_20251027_164240_integral.csv",
    "Tset": 55,
    "band": 1,
```

```
    "control_type": "PI",
    "Offset": 0.165,
    "Kp": 0.3,
    "Ki": 0.00001,
    "notes": "Aluminium"
  },
  {
    "filename": "live_plot_20251027_171632_derivative.csv",
    "Tset": 55,
    "band": 1,
    "control_type": "PID",
    "Offset": 0.165,
    "Kp": 0.3,
    "Ki": 0.00001,
    "Kd": 0.0,
    "notes": "Aluminium"
  },
  {
    "filename": "live_plot_20251028_162354.csv",
    "Tset": 55,
    "band": 1,
    "control_type": "PID",
    "Offset": 0.165,
    "Kp": 0.3,
    "Ki": 0.00001,
    "Kd": 0.00001,
    "notes": "Aluminium"
  },
  {
    "filename": "live_plot_20251029_163401.csv",
    "Tset": 65,
    "band": 1,
    "control_type": "PI",
    "Offset": 0.25,
    "Kp": 0.3,
    "Ki": 0.001,
    "Kd": 0.0,
    "notes": "Aluminium"
  },
  {
    "filename": "live_plot_20251029_170448.csv",
    "Tset": 65,
    "band": 1,
    "control_type": "PID",
    "Offset": 0.25,
    "Kp": 0.3,
    "Ki": 0.001,
    "Kd": 0.001,
    "notes": "Brass"
  }
]
```

Listing 7: PID_data/manifest.json: Json array for PID data files and the respective constants applied to the system.

Arduino Sketches

```
#ifndef CSV_PRINTER_H
#define CSV_PRINTER_H

#include <Arduino.h>

/**
 * @brief A library for printing CSV formatted data to the Arduino Serial monitor.
 *
 * This library provides convenient functions for outputting data in CSV format,
 * making it easy to collect and analyze experimental data from Arduino projects.
 *
 * @author Generated for PHY3902 Labs
 * @version 1.0
 */

class CSVPrinter {
public:
    /**
     * @brief Initialize the CSV printer with Serial communication.
     * @param baudRate The baud rate for Serial communication (default: 9600)
     */
    static void begin(long baudRate = 9600);

    /**
     * @brief Prints a value in CSV format to the Serial output.
     * @tparam T The type of the value to print.
     * @param value The value to print.
     * @param eol If true, prints a newline after the value; otherwise, prints a comma.
     * @param precision Number of decimal places for floating point values (default: 4)
     */
    template <typename T>
    static void printValue(T value, bool eol = false, int precision = 4);

    /**
     * @brief Prints a CSV header row.
     * @param headers Array of header strings
     * @param numHeaders Number of headers in the array
     */
    static void printHeader(const char* headers[], int numHeaders);

    /**
     * @brief Prints a CSV header row using a single string with comma-separated values.
     * @param headerString String containing comma-separated header names
     */
    static void printHeader(const char* headerString);

    /**
     * @brief Starts a new CSV row (no output, just for organization)
```

```
     */
    static void startRow();

    /**
     * @brief Ends the current CSV row with a newline
     */
    static void endRow();

    /**
     * @brief Prints a complete CSV row with 2 values
     */
    template <typename T1, typename T2>
    static void printRow(T1 val1, T2 val2, int precision = 4);

    /**
     * @brief Prints a complete CSV row with 3 values
     */
    template <typename T1, typename T2, typename T3>
    static void printRow(T1 val1, T2 val2, T3 val3, int precision = 4);

    /**
     * @brief Prints a complete CSV row with 4 values
     */
    template <typename T1, typename T2, typename T3, typename T4>
    static void printRow(T1 val1, T2 val2, T3 val3, T4 val4, int
        precision = 4);
};

// Template function implementations


// Generic template for printValue (fallback for types not specialized
    below)
template <typename T>
void CSVPrinter::printValue(T value, bool eol, int /*precision*/) {
    if (eol) {
        Serial.println(value);
    } else {
        Serial.print(value);
        Serial.print(",");
    }
}

// Specialization for float
template <>
inline void CSVPrinter::printValue<float>(float value, bool eol, int
    precision) {
    if (eol) {
        Serial.println(value, precision);
    } else {
        Serial.print(value, precision);
        Serial.print(",");
    }
}

// Specialization for double
template <>
```

```
103 inline void CSVPrinter::printValue<double>(double value, bool eol, int
    precision) {
104     if (eol) {
105         Serial.println(value, precision);
106     } else {
107         Serial.print(value, precision);
108         Serial.print(",");
109     }
110 }
111
112 // Specialization for int
113 template <>
114 inline void CSVPrinter::printValue<int>(int value, bool eol, int /*
    precision*/) {
115     if (eol) {
116         Serial.println(value);
117     } else {
118         Serial.print(value);
119         Serial.print(",");
120     }
121 }
122
123 // Specialization for long
124 template <>
125 inline void CSVPrinter::printValue<long>(long value, bool eol, int /*
    precision*/) {
126     if (eol) {
127         Serial.println(value);
128     } else {
129         Serial.print(value);
130         Serial.print(",");
131     }
132 }
133
134 template <typename T1, typename T2>
135 void CSVPrinter::printRow(T1 val1, T2 val2, int precision) {
136     printValue(val1, false, precision);
137     printValue(val2, true, precision);
138 }
139
140 template <typename T1, typename T2, typename T3>
141 void CSVPrinter::printRow(T1 val1, T2 val2, T3 val3, int precision) {
142     printValue(val1, false, precision);
143     printValue(val2, false, precision);
144     printValue(val3, true, precision);
145 }
146
147 template <typename T1, typename T2, typename T3, typename T4>
148 void CSVPrinter::printRow(T1 val1, T2 val2, T3 val3, T4 val4, int
    precision) {
149     printValue(val1, false, precision);
150     printValue(val2, false, precision);
151     printValue(val3, false, precision);
152     printValue(val4, true, precision);
153 }
154
155 // Non-template function implementations
156
```

```
157 void CSVPrinter::begin(long baudRate) {
158     Serial.begin(baudRate);
159 }
160
161 void CSVPrinter::printHeader(const char* headers[], int numHeaders) {
162     for (int i = 0; i < numHeaders; i++) {
163         Serial.print(headers[i]);
164         if (i < numHeaders - 1) {
165             Serial.print(",");
166         }
167     }
168     Serial.println();
169 }
170
171 void CSVPrinter::printHeader(const char* headerString) {
172     Serial.println(headerString);
173 }
174
175 void CSVPrinter::startRow() {
176     // This function is for organization/readability - no actual output
177 }
178
179 void CSVPrinter::endRow() {
180     Serial.println();
181 }
182
183 #endif // CSV_PRINTER_H
```

Listing 8: CSVPrinter.h: Custom header file to facilitate easy csv output to the serial monitor.

```
1  //**********************************
2  // Arduino ON-OFF temperature control
3  // - Michel Godin
4  //**********************************
5
6  #include "CSVPrinter.h"
7
8  /**
9   * @brief Holds the mean and standard deviation of thermistor readings.
10  */
11 struct Stats {
12     float mean;
13     float stddev;
14 };
15
16 //*** Constants
17 const float Tset = 55.0;     // set temperature in degrees celcius (
       ideally between 40 and 65)
18 const float Band = 1.0;      // Temperature band for hysteresis
19 const float Tmax = 75.0;     // Setting a maximum temperature. For
       safety, heater will be turned off when exceeding this temperature.
20 const int numSamples = 100; // number of samples to average for
       temperature measurement
21 const int gatePin = 9; // PWM pin to control MOSFET
22 const int thermistorPin = A0; // Analog pin connected to the thermistor
23 const long R_series = 100000; // series resistance for thermistor's
       voltage divider
24 const unsigned long RESPONSE_TIME_S = 60;
```

```
25 const unsigned long ActionTime = (unsigned long)(2 * RESPONSE_TIME_S *
       1000); // ActionTime is twice the response time, in ms
26
27 //*** Variable definitions
28 float temp_average; // average measured temperature
29 float Vo; // ADC voltage (digital) measured across thermistor
30 float Rt, lnRt; // thermistor resistance, natural log of thermistor
       resistance
31 float T_kelvin, T_celsius; // temperature (Kelvin), temperature (
       degrees Celsius)
32
33 //*** Variable declarations
34 Stats temp_stats; // struct to hold temperature statistics
35 float A = 0.7141095422e-3, B = 2.143243110e-4, C = 1.129790692e-7; //
       Steinhart-Hart coefficients
36
37 //*** Time associated variable definitions and declarations
38 unsigned long CurrentTime;             // ms
39 unsigned long PreviousTime = 0; // ms
40 unsigned long DeltaTime;                 // ms
41 unsigned long MaxTime = 30;    // Time limit: Maximum duration of
       experiment in minutes until the heater is turned OFF.
42 unsigned long LastIntegralUpdate = 0; // Last time the integral was
       updated
43 unsigned long windowStartTime = 0;
44
45
46 //*** Some variables that are not used now, but might become useful
       when you implement PID control
47 float Error;        // The error (Tset - temp_average)
48 float Offset = 0.165; // offset for proportional control
49 float Previous_Error = 0.0; // necessary for the PID controller when
       calculating the integral and derivative
50 float Kp = 0.3; // proportional gain - you can tune this value
51 float Ki = 0.00001; // integral gain, can be zero
52 float Integral = 0.0; // Integral value: sum of the areas
53 int Power;        // When implementing PID, Power will be calcuated and
       this value will be the PWM output between 0-255)
54                           // Power = (Kp*Error + Ki*Integral + Kd*
                                Derivative) * 255;
55
56 void setup() {
57     pinMode(gatePin, OUTPUT);
58     CSVPrinter::begin(9600);
59     Serial.println("ON-OFF␣Control");
60     Serial.println("---DATA␣START---");
61     CSVPrinter::printHeader("time(s),temp_mean(C),temp_stddev(C),
           output_power");
62 }
63
64 void loop() {
65     get_time();
66     temp_stats = measureTemperature(thermistorPin, numSamples);
67
68     Error = (Tset - temp_stats.mean) / Band;
69
70     // Reset Integral at the start of each ActionTime window
71     if (CurrentTime - windowStartTime >= ActionTime) {
```

```
72          Integral = 0.0;
73          windowStartTime = CurrentTime;
74      }
75
76      // Accumulate error * dt within the window
77      float dt = DeltaTime / 1000.0; // seconds
78      Integral += Error * dt;
79
80      integrate(Power, Tset, temp_stats, Offset, Kp);
81      analogWrite(gatePin, Power);
82
83      CSVPrinter::printRow(CurrentTime/1000.0, temp_stats.mean,
            temp_stats.stddev, Power, 3);
84      delay(10);
85  }
86
87
88  // Below, we define several functions that are used in the main LOOP()
89  // We use function to make the main LOOP() a little tidier
90
91  //* This function computes the average temperature for better results
92  Stats measureTemperature(int pin, int numSamples) {
93      float temp_sum = 0.0;
94      float temp_squared_sum = 0.0;
95      float Vo, Rt, lnRt, T_kelvin, T_celsius;
96      Stats stats;
97
98      for (int m = 0; m < numSamples; m++) {
99          Vo = analogRead(pin);
100         Rt = R_series * (Vo / (1023.0 - Vo));
101         lnRt = log(Rt);
102         T_kelvin = (1.0 / (A + B * lnRt + C * pow(lnRt, 3)));
103         T_celsius = T_kelvin - 273.15;
104         temp_sum += T_celsius;
105         temp_squared_sum += T_celsius * T_celsius;
106         delay(10);
107     }
108     stats.mean = temp_sum / numSamples;
109     float variance = (temp_squared_sum - numSamples * stats.mean *
            stats.mean) / (numSamples - 1);
110     stats.stddev = sqrt(abs(variance));
111     return stats;
112
113 // For safety: The following turns off the heater if the temperature
        exceed the pre-defined MAX temperature
114     // if(temp_average >= Tmax){
115     //     while(1){                 // Notice that this while
            condition is always true, so this will permanently stop the code
             until you reset the Arduino
116     //         analogWrite(gatePin,0);     //turning off the heater
117     //         Serial.println("Maximum temperature has been exceeded.
            Heater is turned OFF. Reset Arduino to restart the program");
118     //         delay(5000);
119     //     }
120     // }
121 }
122
123 void integrate(float power, float Tset, Stats temp_stats, float offset,
```

```
    float Kp){
    Previous_Error = Error; // update previous error
    int out = (offset + Error * Kp + Integral * Ki) * 255;
    Power = constrain(out, 0, 255);
}

//* This function keeps track of the time.
void get_time(){
    CurrentTime = millis();         // Time elapsed (in milliseconds)
       since the beginning
    DeltaTime = CurrentTime - PreviousTime;    // This "might" be
       useful when you implement integral or derivative control!
    PreviousTime = CurrentTime; // ready for next loop!

// For safety: The following IF statement prevents any experiment from
   lasting longer than "MaxTime" minutes
    if(CurrentTime >= MaxTime*60*1000){             // converting
       MaxTime from minutes to milliseconds
        while(1){
            analogWrite(gatePin,0);     //turning off the heater
            Serial.println("Maximum time has been reached, heater is
               turned OFF. Reset Arduino to restart the program");
            delay(5000);
        }
    }
}
```

Listing 9: Temp_ON_OFF_integral.ino: Full PID control for Arduino Uno.

```
//**********************************
// Simple Arduino Step Response Measurement
//**********************************

const int gatePin = 9; // PWM pin to control MOSFET
const int thermistorPin = A0; // Analog pin connected to the thermistor

unsigned long stepTime = 0;
unsigned long responseTime = 0;
bool stepApplied = false;
float initialTemp = 0.0;
float targetTemp = 0.0;

// Variables for timing and state
unsigned long stepTime = 0;      // Time when step input (heater ON) is
    applied
unsigned long responseTime = 0;  // Time taken to reach 63% of
   temperature step
bool stepApplied = false;        // Flag to indicate if step has been
   applied
float initialTemp = 0.0;         // Initial temperature before step
float targetTemp = 0.0;          // Target temperature (step = initial
   + 5C)

// Reads temperature in Celsius from thermistor using Steinhart-Hart
   equation
float readTemperatureC(int pin) {
    float Vo = analogRead(pin);
    float Rt = R_series * (Vo / (1023.0 - Vo));
```

```
    float lnRt = log(Rt);
    float T_kelvin = (1.0 / (A + B * lnRt + C * pow(lnRt, 3)));
    return T_kelvin - 273.15;
}

void setup() {
    pinMode(gatePin, OUTPUT);
    Serial.begin(9600);
    delay(1000);
    Serial.println("Simple Step Response Measurement");
    // Measure initial temperature (average 10 samples)
    float sum = 0.0;
    for (int i = 0; i < 10; i++) {
        sum += readTemperatureC(thermistorPin);
        delay(100);
    }
    initialTemp = sum / 10.0;
    targetTemp = initialTemp + 5.0; // Define step: 5C above initial
    Serial.print("Initial temperature: ");
    Serial.print(initialTemp, 2);
    Serial.println(" C");
    Serial.print("Target temperature (step): ");
    Serial.print(targetTemp, 2);
    Serial.println(" C");
    Serial.println("Applying step (heater ON)...");
    analogWrite(gatePin, 255); // Apply step: turn heater fully ON
    stepTime = millis();       // Record time of step
    stepApplied = true;        // Mark that step has been applied
}

void loop() {
    if (!stepApplied) return;
    float temp = readTemperatureC(thermistorPin);
    unsigned long now = millis();
    float elapsed = (now - stepTime) / 1000.0; // Time since step (
       seconds)
    Serial.print("Time: ");
    Serial.print(elapsed, 1);
    Serial.print(" s, Temp: ");
    Serial.print(temp, 2);
    Serial.println(" C");

    // Calculate 63% threshold for first-order response (time constant)
    // threshold = initial + 0.63 * (target - initial)
    float threshold = initialTemp + 0.63 * (targetTemp - initialTemp);
    if (temp >= threshold) {
        responseTime = now - stepTime;
        Serial.println();
        Serial.print("Response time (63% rise): ");
        Serial.print(responseTime / 1000.0, 2);
        Serial.println(" s");
        analogWrite(gatePin, 0); // Turn off heater
        Serial.println("Heater OFF. Done. Reset Arduino to repeat
           measurement.");
        stepApplied = false; // Stop further measurements
    }
    delay(1000);
}
```

Listing 10: response_time_measurement.ino: Simple Arduino sketch to determine the response time of a heating system.

```
// Arduino code to experimentally determine the specific heat capacity of a metal block
// Heater is controlled via PWM on pin 9, thermistor is read from A0
// Data is output as CSV: time(s), temp(C), voltage(V), current(A), power(W)

#include "CSVPrinter.h"

const float R_heater = 15.7; // resistance of the heater in ohms
const float R_series = 100000.0; // series resistor for thermistor voltage divider
const int heaterPin = 9; // PWM pin for heater
const int thermistorPin = A0; // Analog pin for thermistor
const float V_supply = 12.0; // Supply voltage (V)
const int numSamples = 100; // Number of samples for temperature averaging
const int pwmValue = 255; // PWM value (0-255) for heater

// Steinhart-Hart coefficients for thermistor
const float A = 0.7141095422e-3, B = 2.143243110e-4, C = 1.129790692e-7;

struct Stats {
    float mean;
    float stddev;
};

unsigned long startTime;

// Cycle control variables
const unsigned long interval_ms = 5UL * 60UL * 1000UL; // 2 minutes in ms
int phase = 0; // 0: heating, 1: cooling
int cycle_count = 0;
const int max_cycles = 2; // 2 heating + 2 cooling phases
unsigned long phase_start_time = 0;
bool finished = false;

void setup() {
    pinMode(heaterPin, OUTPUT);
    CSVPrinter::begin(9600);
    Serial.println("Specific Heat Capacity Experiment");
    Serial.println("---DATA START---");
    CSVPrinter::printHeader("time(s),temp_mean(C),temp_stddev(C),
        voltage(V),current(A),power(W)");
    startTime = millis();
    phase_start_time = millis();
}

void loop() {
    if (finished) {
        analogWrite(heaterPin, 0); // Ensure heater is off
        delay(1000);
        return;
```

```
    }

    unsigned long now = millis();
    // Determine if we need to switch phase
    if ((now - phase_start_time) >= interval_ms) {
        phase_start_time = now;
        phase = 1 - phase; // Toggle phase: 0 <-> 1
        cycle_count++;
        if (cycle_count >= 2 * max_cycles) {
            finished = true;
            analogWrite(heaterPin, 0); // Turn off heater
        }
    }

    // Set heater state
    if (!finished && phase == 0) {
        analogWrite(heaterPin, pwmValue); // Heating
    } else {
        analogWrite(heaterPin, 0); // Cooling
    }

    // Measure temperature
    Stats temp_stats = measureTemperature(thermistorPin, numSamples);

    // Calculate heater voltage and current
    float V_heater = (phase == 0 && !finished) ? (V_supply * pwmValue /
        255.0) : 0.0;
    float I_heater = (phase == 0 && !finished) ? (V_heater / R_heater)
       : 0.0;
    float P_heater = (phase == 0 && !finished) ? (V_heater * I_heater)
       : 0.0;

    // Log data
    float t = (now - startTime) / 1000.0;
    CSVPrinter::printRow(t, temp_stats.mean, temp_stats.stddev,
       V_heater, I_heater, P_heater, 3);

    delay(10); // Log every 10 ms
}

Stats measureTemperature(int pin, int n) {
    float temp_sum = 0.0, temp_sq_sum = 0.0;
    for (int i = 0; i < n; i++) {
        float Vo = analogRead(pin);
        float Rt = R_series * (Vo / (1023.0 - Vo));
        float lnRt = log(Rt);
        float T_kelvin = 1.0 / (A + B * lnRt + C * pow(lnRt, 3));
        float T_celsius = T_kelvin - 273.15;
        temp_sum += T_celsius;
        temp_sq_sum += T_celsius * T_celsius;
        delay(5);
    }
    Stats stats;
    stats.mean = temp_sum / n;
    float variance = (temp_sq_sum - n * stats.mean * stats.mean) / (n -
        1);
    stats.stddev = sqrt(abs(variance));
    return stats;
```

```
101 }
```

Listing 11: specific_heat_measurement.ino: Simple Arduino sketch to apply constant heating cycles to a heater.